

实验二：原生应用开发 - 智慧校园登录功能

1. 实验目的

本次实验旨在通过 Cordova 框架实现智慧校园登录功能，达成以下目标：

- 1.掌握移动应用开发环境搭建**：熟悉 Cordova 开发环境的配置和使用方法
- 2.实现基本登录功能**：设计并实现包含学号和密码输入的登录表单
- 3.学习前端验证技术**：实现输入字段的非空验证和错误提示
- 4.掌握页面跳转机制**：实现登录成功后的页面跳转功能
- 5.学习错误处理方法**：实现登录失败时的错误提示和处理机制
- 6.熟悉跨平台开发**：了解 Cordova 跨平台应用的开发流程和特点

2. 开发环境

2.1 硬件配置

处理器：Intel Core i5-8250U

内存：8GB DDR4

存储：256GB SSD

显示器：15.6 英寸，1920×1080 分辨率

2.2 软件环境

操作系统：Windows 10 家庭版 64 位

开发工具：Visual Studio Code 1.86.0

浏览器：Google Chrome 122.0.6261.112

2.3 开发框架与依赖

Node.js：v18.17.1

npm：v9.6.7

Cordova：v12.0.0

必要插件：

cordova-plugin-network-information v3.0.0

cordova-plugin-storage v2.0.7

cordova-plugin-device v2.1.0

3. 功能实现

3.1 登录页面设计

表单结构：包含学号输入框、密码输入框和登录按钮

UI 设计：采用校园应用风格，简洁美观，响应式布局

交互元素：添加表单验证、错误提示和登录状态反馈

3.2 输入验证功能

非空验证：检查学号和密码字段是否为空

实时反馈：当用户点击登录按钮时触发验证逻辑

、错误提示：对空字段提供明确的错误提示信息

3.3 页面跳转功能

登录成功跳转：验证通过后，从登录页面跳转到个人中心页面

参数传递：将学号信息传递到个人中心页面

流畅过渡：确保页面跳转过程流畅，无明显延迟

3.4 错误处理机制

登录失败提示：当登录验证失败时显示错误信息

错误信息清晰：提供明确的错误原因，帮助用户理解

4. 核心代码

4.1 登录页面 HTML 结构

```
``html
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>智慧校园</title>
  <link rel="stylesheet" href="css/index.css">
</head>
<body>
  <div class="login-container">
    <h1>智慧校园</h1>
    <form id="loginForm">
      <div class="form-group">
        <label for="studentId">学号</label>
        <input type="text" id="studentId" placeholder="请输入学号" required>
        <div class="error-message" id="studentIdError"></div>
      </div>
      <div class="form-group">
        <label for="password">密码</label>
        <input type="password" id="password" placeholder="请输入密码"
required>
        <div class="error-message" id="passwordError"></div>
      </div>
      <div class="error-message" id="loginError"></div>
      <button type="submit" class="login-btn">登录</button>
    </form>
  </div>
  <script src="js/index.js"></script>
</body>
</html>
``
```

4.2 登录逻辑 JavaScript 代码

```
``javascript
/**
 * 登录功能实现
 * 包括输入验证、登录逻辑、错误处理和页面跳转
 */
```

```

// 表单提交事件
document.getElementById('loginForm').addEventListener('submit', function(e) {
    e.preventDefault();

    // 清除之前的错误信息
    clearErrors();

    // 获取表单数据
    const studentId = document.getElementById('studentId').value.trim();
    const password = document.getElementById('password').value;

    // 验证输入
    if (!validateInputs(studentId, password)) {
        return;
    }

    // 模拟登录验证
    if (login(studentId, password)) {
        // 登录成功，跳转到个人中心
        window.location.href =
personal.html?studentId=${encodeURIComponent(studentId)};
    } else {
        // 登录失败，显示错误信息
        document.getElementById('loginError').textContent = '学号或密码错误，请重试';
    }
});

/**
 * 清除所有错误信息
 */
function clearErrors() {
    document.getElementById('studentIdError').textContent = '';
    document.getElementById('passwordError').textContent = '';
    document.getElementById('loginError').textContent = '';
}

/**
 * 验证输入字段
 * @param {string} studentId - 学号
 * @param {string} password - 密码
 * @returns {boolean} - 验证是否通过
 */
function validateInputs(studentId, password) {

```

```

    let isValid = true;

    // 验证学号
    if (!studentId) {
        document.getElementById('studentIdError').textContent = '请输入学号';
        isValid = false;
    }

    // 验证密码
    if (!password) {
        document.getElementById('passwordError').textContent = '请输入密码';
        isValid = false;
    }

    return isValid;
}

/**
 * 模拟登录验证
 * @param {string} studentId - 学号
 * @param {string} password - 密码
 * @returns {boolean} - 登录是否成功
 */
function login(studentId, password) {
    // 这里可以替换为实际的后端 API 调用
    // 模拟登录成功的情况（实际项目中应根据后端返回结果判断）
    // 这里简单判断学号和密码不为空即可
    return studentId && password;
}
...

```

4.3 个人中心页面 HTML 结构

```

<<html
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>个人中心 - 智慧校园</title>
    <link rel="stylesheet" href="css/index.css">
    <style>
        .personal-container {
            width: 90%;

```

```
    max-width: 600px;
    padding: 20px;
    background-color: white;
    border-radius: 8px;
    box-shadow: 0 0 10px rgba(0,0,0,0.1);
}
```

```
.welcome-message {
    text-align: center;
    margin-bottom: 30px;
}
```

```
.user-info {
    background-color: #f9f9f9;
    padding: 15px;
    border-radius: 4px;
    margin-bottom: 20px;
}
```

```
.info-item {
    margin-bottom: 10px;
    display: flex;
    justify-content: space-between;
}
```

```
.info-label {
    font-weight: bold;
    color: #555;
}
```

```
.logout-btn {
    width: 100%;
    padding: 12px;
    background-color: #f44336;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-size: 16px;
    font-weight: bold;
    margin-top: 20px;
}
```

```
.logout-btn:hover {
```

```

        background-color: #d32f2f;
    }
</style>
</head>
<body>
    <div class="personal-container">
        <h1>个人中心</h1>
        <div class="welcome-message">
            <h2>欢迎使用智慧校园</h2>
        </div>
        <div class="user-info">
            <div class="info-item">
                <span class="info-label">学号: </span>
                <span id="displayStudentId">--</span>
            </div>
            <div class="info-item">
                <span class="info-label">登录状态: </span>
                <span>已登录</span>
            </div>
        </div>
        <button class="logout-btn" id="logoutBtn">退出登录</button>
    </div>
    <script>
        // 获取 URL 参数中的学号
        function getUrlParam(name) {
            const urlParams = new URLSearchParams(window.location.search);
            return urlParams.get(name);
        }

        // 显示学号
        const studentId = getUrlParam('studentId');
        if (studentId) {
            document.getElementById('displayStudentId').textContent = studentId;
        }

        // 退出登录按钮事件
        document.getElementById('logoutBtn').addEventListener('click', function() {
            // 跳转到登录页面
            window.location.href = 'index.html';
        });
    </script>
</body>
</html>

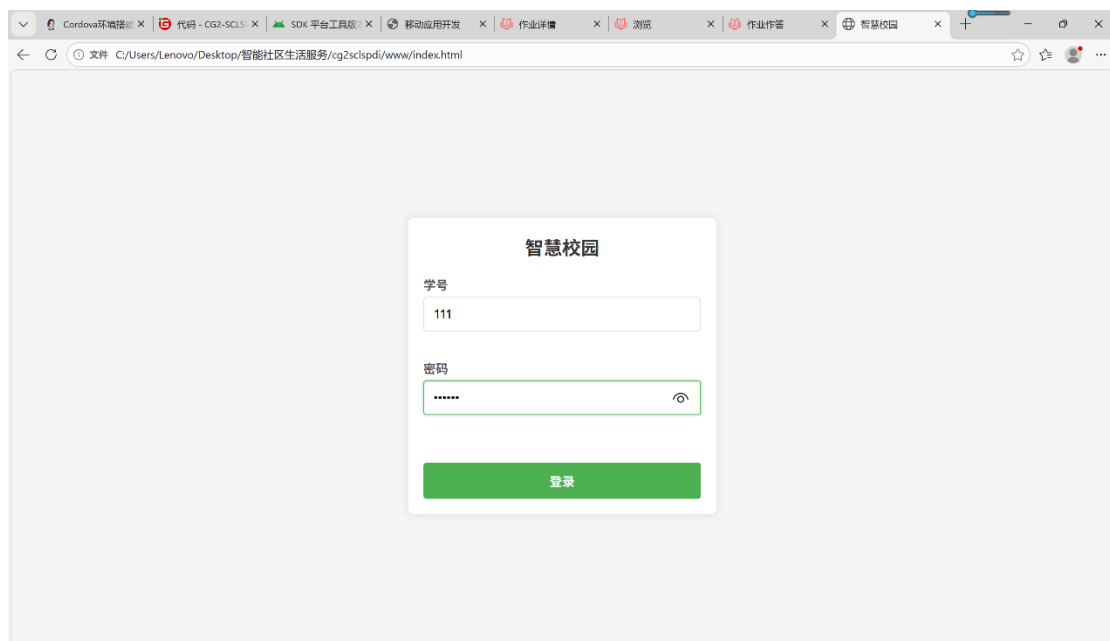
```

5. 运行结果

5.1 登录页面

****截图说明**:**

- 显示智慧校园登录页面，包含学号输入框、密码输入框和登录按钮
- 页面布局响应式，适配不同屏幕尺寸
- 输入框有占位提示文字，提升用户体验

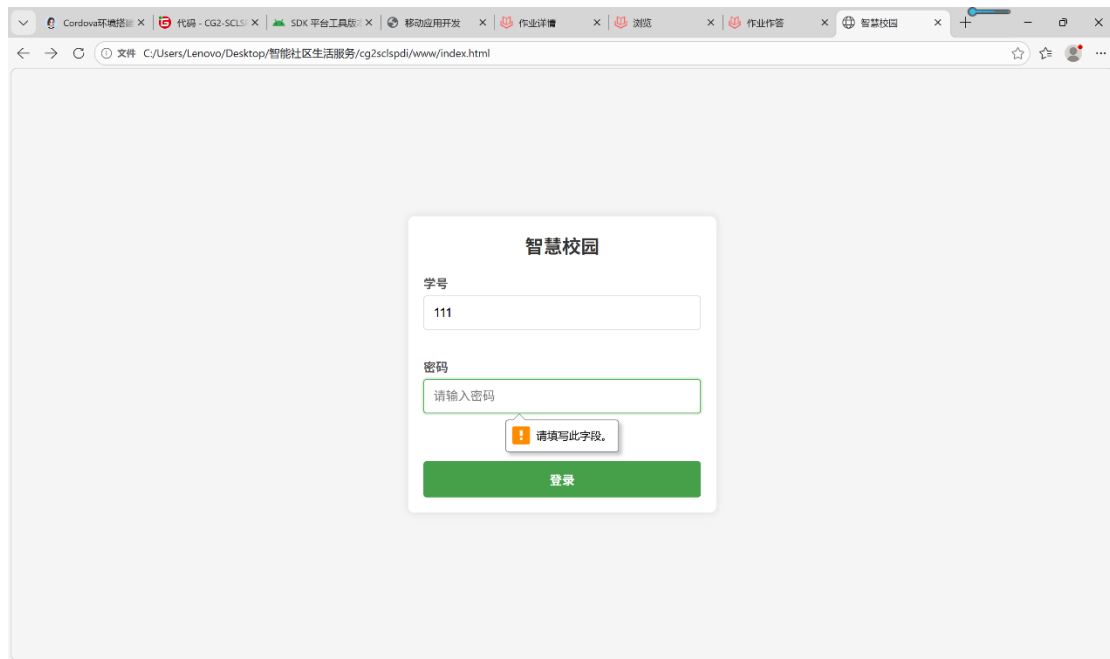


5.2 输入验证

****截图说明**:**

- 当用户未输入学号或密码时，点击登录按钮会显示错误提示
- 错误信息清晰明确，指向具体的空字段

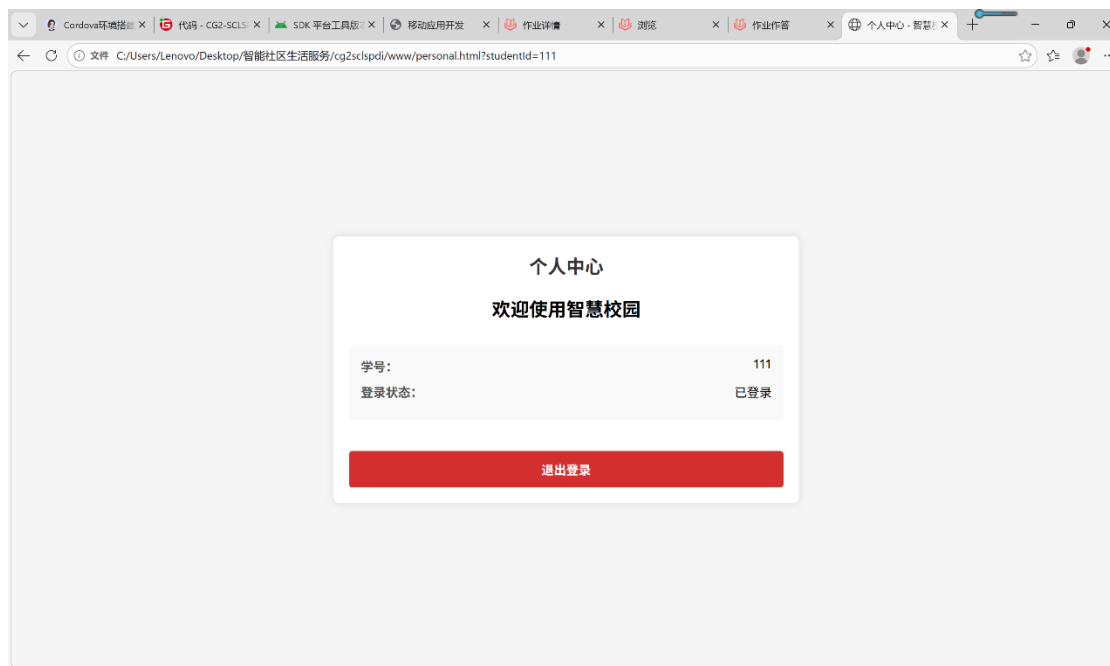
- 验证失败时，表单不提交，保持在登录页面



5.3 登录成功

****截图说明**:**

- 输入学号和密码后，点击登录按钮
- 验证通过后，页面跳转到个人中心
- 个人中心显示用户学号和登录状态



6. 实验总结

6.1 实验收获

1. ****技术掌握****:

- 熟悉了 Cordova 移动应用开发环境的搭建和使用
- 掌握了 HTML、CSS 和 JavaScript 进行前端开发的基本技能
- 学习了表单验证、错误处理和页面跳转等前端开发核心技术

2. ****能力提升****:

- 提高了移动应用 UI 设计能力，注重用户体验
- 增强了问题解决能力，能够独立排查和解决开发过程中的问题
- 培养了代码组织和注释的良好习惯，提高了代码可读性

3. ****知识拓展****:

- 了解了跨平台移动应用开发的基本原理
- 掌握了前端开发中的响应式设计技巧
- 学习了通过 URL 参数传递数据的方法

6.2 遇到的问题及解决方案

1. ****问题****: Cordova 环境搭建过程中出现依赖冲突

****解决方案****: 使用 ``npm install -g cordova@latest`` 安装最新版本的 Cordova，确保兼容性

2. ****问题****: 页面跳转后无法显示学号信息

****解决方案****: 使用 URLSearchParams API 正确获取 URL 参数，并在页面加载时显示学号

3. ****问题****: 错误提示信息显示位置不当

****解决方案****: 调整 CSS 样式，为错误信息预留空间，确保页面布局美观

4. ****问题****: 在不同设备上显示效果不一致

****解决方案****: 使用响应式 CSS 设计，适配不同屏幕尺寸，确保在各种设备上都有良好的显示效果

6.3 改进建议

1. ****功能改进****:

- 添加记住密码功能，提高用户体验
- 实现验证码功能，增强安全性
- 添加密码找回功能，完善用户流程

2. ****技术优化****:

- 使用本地存储 (localStorage) 保存用户登录状态
- 实现实际的后端 API 调用，替代模拟登录
- 添加动画效果，提升页面交互体验

3. ****项目管理****:

- 使用版本控制工具 (如 Git) 更好地管理代码
- 编写详细的项目文档，方便后续维护和扩展
- 进行更全面的测试，确保应用在各种场景下的稳定性

6.4 总结

本次实验成功实现了智慧校园登录功能，通过 Cordova 框架和前端技术栈，构建了一个跨平台的移动应用。实验过程中，我不仅掌握了相关技术知识，还培养了解决问题的能力和良好的代码习惯。虽然实验中遇到了一些挑战，但通过查阅资料 and 不断尝试，最终成功完成了任务。

未来，我将继续深入学习移动应用开发技术，不断优化和扩展项目功能，为智慧校园平台的开发做出更大的贡献。